

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student Full Name	Sherekhan Marzhan
Student Identification Number	230109015
Date & Laboratory Session	18.09.2026
Workstation / Laptop Make & Model	Windows 11 AMD64 12th Gen Intel Core i7-12700H
Processor SKU / SoC Model	e.g., AMD Ryzen 7 5800H / Apple M2 Pro / Intel Core i7-12700H
Core Topology (Physical P-cores / E-cores)	Physical Cores: __14__ Logical SMT Threads: __20__
Cache Hierarchy Capacity	L1: ____ KB L2: ____ MB L3: ____ MB
Host OS & Python Runtime	e.g., Linux Kernel 6.8 / Ubuntu 24.04 LTS / CPython 3.11.8

■ MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where $S(p)$ is the speedup ratio $T(1)/T(p)$, P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■ EXECUTION PROTOCOL

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1_amdahl.py'. Record the single-process baseline T(1) and the measured multi-core execution times across all 6 core allocations.

```
import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f'{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x')
```

p = 1 (Baseline)	1.6312	1.00x	1.00x	100.0%
p = 2	0.8685	2.29x	2.00x	
p = 4	0.5286	3.76x	4.00x	

p = 8	0.5022	3.96×	8.00×	
p = 12	0.5284	3.76×	12.00×	
p = 16	0.6221	3.20×	16.00×	

[Q1.1] Identify the exact inflection point (core count p^*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). (8 Points)

Student Response & Architectural Justification: The best measured time occurs at 8 workers: 0.5022 s.

After that:

- 8 → 12: 0.5022 → 0.5284 s
- 8 → 16: 0.5022 → 0.6221 s

Therefore the empirical inflection point is:

$p^*=8$

The observed speedup curve improves strongly from 1 to 8 workers, reaching its minimum execution time of 0.5022 s at 8 workers. After 8 workers, performance degrades: execution time increases to 0.5284 s at 12 workers and 0.6221 s at 16 workers. Therefore, the empirical inflection point is $p^* = 8$. My CPU has 14 physical cores and 20 logical processors. The degradation occurs before all 14 physical cores are fully represented by the tested worker counts, indicating that process overhead, CPU resource contention, cache effects, and frequency/thermal behavior limit scaling. Logical SMT threads would provide additional concurrency but do not provide the same resources as independent physical cores.

[Q1.2] Using your empirical measurements for $T(1)$ and $T(4)$, calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$. (9 Points)

*Student Response & Architectural Justification:*We have:
 $T(1)=1.9888$ $T(1)=1.9888$ $T(4)=0.5286$ $T(4)=0.5286$

Therefore:

$$S(4)=T(1)/T(4)=\frac{T(1)}{T(4)} \quad S(4)=1.9888/0.5286 \quad S(4)=\frac{1.9888}{0.5286} \quad S(4)=3.7624 \quad \boxed{S(4)=3.7624}$$

Amdahl's equation:

$$S(4)=1/(1-P)+P/4 \quad S(4)=\frac{1}{(1-P)+P/4}$$

Substitute:

$$3.7624=1/(1-P)+P/4 \quad 3.7624=\frac{1}{1-P+\frac{P}{4}} \quad 3.7624=1/(1-\frac{3P}{4}) \quad 3.7624=\frac{1}{1-\frac{3P}{4}}$$

Take reciprocal:

$$13.7624=1-3P \quad \frac{1}{3.7624}=1-\frac{3P}{4} \quad 0.26579=1-3P \quad 0.26579=1-\frac{3P}{4} \quad 3P=0.73421 \quad \frac{3P}{4}=0.73421/3 \quad P=0.97895 \quad P=0.97895$$

Therefore:

$$P=97.895\%$$

Student Response & Architectural Justification:

The measured speedup at four workers is

$$S(4)=T(1)/T(4)=1.9888/0.5286=3.7624 \quad S(4)=T(1)/T(4)=1.9888/0.5286=3.7624.$$

Using Amdahl's Law:

$$3.7624=1/(1-P)+P/4 \quad 3.7624=\frac{1}{(1-P)+P/4}$$

Therefore,

$$13.7624=1-3P \quad \frac{1}{3.7624}=1-\frac{3P}{4}$$

which gives $P=0.97895$. Thus, the empirically estimated parallel fraction is approximately **97.895%**.

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim_{p \rightarrow \infty} S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload. (8 Points)

*Student Response & Architectural Justification:*Formula:
 $S_{\max}=1/(1-P)$

Using:

$$P=0.97895 \quad P=0.97895 \quad S_{\max}=1/(1-0.97895) \quad S_{\max}=\frac{1}{1-0.97895} \quad S_{\max}=47.50 \times \quad \boxed{S_{\max} \approx 47.50 \times}$$

Worksheet answer

Student Response & Architectural Justification:

Using the measured parallel fraction $P=0.97895$

$$S_{max} = \frac{1}{1-P} \quad S_{max} = \frac{1}{1-0.97895} \approx 47.50$$

Therefore, the theoretical asymptotic speedup is approximately 47.5x. A 128-core server would not provide 128x speedup for this workload. According to the Amdahl model using the measured P, speedup would asymptotically approach approximately 47.5x, while real hardware overheads would generally make actual performance lower.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements (16 * 8 bytes = 128 bytes, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```

import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")

```

Adjacent Indices (False Sharing Active)	0.7316	0.6482	0.7761	0.7316 s	— [Baseline]
Padded Indices (Cache-Line Aligned)	0.6775	0.7138	0.5921	0.6775 s	

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8 Points)

Student Response & Architectural Justification: On a 64-bit CPython build:
 $1 \text{ pointer} = 8 \text{ bytes}$

The padded version uses:

$16 \times 8 = 128 \text{ bytes}$

A cache line is:

64 bytes

Therefore:

$128 > 64$

and two corresponding padded accesses are separated by at least two cache-line widths.

Put this:

Student Response & Architectural Justification:

On a 64-bit CPython build, each list element is represented in the list's pointer array by an 8-byte reference. The padded index stride is 16 elements, so the reference-array distance is:

$16 \times 8 = 128 \text{ bytes}$

Since a cache line is 64 bytes, 128 bytes corresponds to two cache-line widths. Thus, two padded indices cannot occupy the same 64-byte cache line. The adjacent configuration uses consecutive 8-byte references, so four consecutive references occupy only 32 bytes and can reside within the same cache line depending on alignment, creating the possibility of false sharing.

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid). (9 Points)

Student Response & Architectural Justification: Initially, the cache line containing the shared array elements may be in the Shared state if multiple cores have a copy. When Core 0 writes to index 0, it requires exclusive ownership of the cache line, causing coherence invalidation of other cached copies. The line transitions toward the Modified state in Core 0. If Core 1 then writes to index 1 on the same cache line, Core 1 must obtain exclusive ownership, causing Core 0's copy to become Invalid. Core 0 subsequently needs to reacquire the line. Repeated alternating writes therefore cause cache-line ownership transfers and invalidations. This coherence traffic is the mechanism behind false-sharing overhead.

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding). (8 Points)

Student Response & Architectural Justification: One industry-standard mechanism is C++11 `alignas(64)`, which allows a data object or structure to be explicitly aligned to a 64-byte cache-line boundary. Another mechanism is Java's `@Contended` annotation, which adds VM-level padding around contended fields to prevent independent variables from sharing cache lines. Go programs can similarly use explicit struct padding to separate frequently modified fields. These techniques prevent unrelated frequently written variables from occupying the same cache line.

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time
```

```
TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

```

class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")

```

Unsafe Accumulator (Race Condition)	2,000,000				1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)		

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)

Student Response & Architectural Justification:

Unsafe: 2,000,000 / 2,000,000

Time: 0.1356 s

Locked: 2,000,000 / 2,000,000

Time: 0.3104 s

Contention multiplier: 2.29×

The worksheet describes *UnsafeCounter* as causing corruption, but your CPython 3.14 experiment did not produce corruption.

Your dedicated forensic experiment confirms this:

- 5/5 normal trials: 2,000,000
- 5/5 forced-switch trials: 2,000,000
- Instrumented version with an actual preemption point: corruption occurred in all 5 trials.

This is actually a valuable empirical result. **Do not write a fake corrupted number.**

■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly FASTER than LockedCounter by at least 2.0x. Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter. (11 Points)

Student Response & Architectural Justification:

Your measured result:

$T_{\text{locked}}=0.3002s$ $T_{\text{lockless}}=0.0410s$

Therefore:

$\text{Speedup} = \frac{0.3002}{0.0410} = 7.32 \times$

Final result:

2,000,000,000,000

Correctness: **PASS**

Performance requirement $\geq 2\times$: **PASS**

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)

Student Response & Architectural Justification:

Eliminating shared mutable state removes the need for threads to repeatedly compete for the same synchronization primitive. With thread-local buffers, each thread can update its own state independently, avoiding lock acquisition, lock release, cache-line contention, and serialization of the critical section. A final reduction combines the independent results after the parallel work is complete. Therefore, instead of making a shared lock more efficient, the architecture removes the source of contention itself. This is particularly useful in high-throughput systems where frequent synchronization would otherwise limit scalability.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```

import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()

```

w = 1 Worker (Baseline)		1.00x		1.00x
w = 2 Workers				

w = 4 Workers				

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

Student Response & Architectural Justification:

The compute-bound workload remains approximately constant in elapsed time as worker count increases: 0.8128 s with one worker, 0.7881 s with two workers, and 0.8372 s with four workers. This indicates that additional parallel workers do not provide proportional wall-clock improvement because the workload is dominated by CPU execution resources and overhead.

The memory-bound workload behaves differently. Its execution time increases from 0.5620 s with one worker to 0.6238 s with two workers and 0.9877 s with four workers. The four-worker execution therefore takes approximately 1.76× the one-worker time. Multiple processes compete for memory bandwidth, increasing memory-controller and DRAM contention. Once the memory subsystem becomes saturated, additional workers increase contention rather than increasing useful throughput.

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

Student Response & Architectural Justification:

- 6400 MT/s
- 16 GB total
- SMBIOS memory type 35
- eight reported 2 GB memory entries.

The worksheet provides:

$$\text{Peak GB/s} = \frac{\text{MT/s} \times \text{bus width (bits)} / 8 \times \text{channels}}{1000}$$

For a 128-bit total memory interface, which corresponds to two 64-bit channels:

$$6400 \times 128 / 1000 = 6400 \times \frac{128}{8} / 1000 = 6400 \times 16 / 1000 = 6400 \times 16 / 1000 = 102.4 \text{ GB/s}$$

Put this:

Student Response & Architectural Justification:

The hardware report shows a memory speed of 6400 MT/s and a total capacity of 16 GB. Assuming the memory interface provides a total 128-bit data path (two 64-bit channels), the theoretical peak bandwidth is:

$$\text{Peak} = 6400 \times 128 / 8 / 1000 = 102.4 \text{ GB/s}$$

This is a theoretical peak, not the measured application bandwidth. The memory bus limits scalability because all CPU cores share the available DRAM bandwidth. When multiple workers stream large arrays simultaneously, they compete for the same memory-controller resources. Once that bandwidth approaches saturation, adding CPU workers cannot proportionally increase throughput.

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature

vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is memory-bandwidth saturated? Justify your architectural choice. (8 Points)

Student Response & Architectural Justification:

If the embedding workload is already saturated by memory bandwidth, simply adding more CPU workers or OpenMP threads is unlikely to provide proportional speedup because the limiting resource is the memory subsystem rather than the number of CPU execution units. CUDA could provide a larger benefit only if the GPU has substantially greater usable memory bandwidth and the workload can efficiently use it. However, transferring data between CPU and GPU can introduce additional overhead. Therefore, the architectural decision should be based on the available memory bandwidth and data-movement cost, rather than assuming that GPU computation is automatically faster. For a genuinely memory-bandwidth-bound pipeline, a device with higher effective memory bandwidth is the relevant optimization target.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
[] Task 3 Unsafe Corruption Verified [] Task 2 Cache Slowdown Verified [] Terminal Logfile 'results_<id>.txt' Uploaded	Examiner Signature: _____ Date:

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

Linux (Ubuntu/Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1dcachesize hw.l2cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

import time

from multiprocessing import Pool

import threading

def cpu_work(n):

Pure CPU crunch

count = 0

for i in range(n):

count += i * i

return count

Run both experiments with total_work = 50_000_000 split across T

workers # Test T in [1, 2, 4, 8, 16, 32]

Deliverables:

- The Empirical Curve:** Plot $T \times S$ (workers) on the X-axis vs. Execution Time on the Y axis for both threading.Thread and multiprocessing.Pool.
- The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
- The Architectural Autopsy:**
 - Why does Python's threading curve remain virtually flat or get worse, while

- multiprocessing scales? (Forces them to articulate the GIL).
- o Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*? o Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T > \text{physical cores}$ using context-switching overhead.

The threading curve is basically flat around 1.8–1.9 s, while multiprocessing improves substantially until 8 processes.

The measured process minimum is:

$$T=8\boxed{T=8}$$

At 16 workers:

$$0.6366 > 0.6177 \quad 0.6366 > 0.6177$$

so the first measured deterioration after the minimum is:

$$T=16$$